Fork me on GitHub

- Home
- What is Flux?
- Guides
  - Installation
  - Using with React
  - Quick-Start Guide
  - Dealing with Asynchronous Data
  - Routing
- Examples
  - Basic Todos
  - Image Carousel
  - Asynchronous Data
  - React Router
- Documentation
  - Fluxxor.Flux
  - Stores
  - Actions
  - Fluxxor.FluxMixin
  - Fluxxor.StoreWatchMixin
- FAQ
- Changelog
- View on GitHub

Support and chat on Gitter
chat on gitter

Copyright © 2014 Michelle Tilley
Licensed under the MIT license

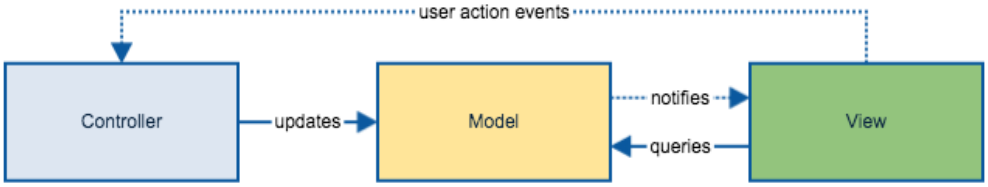
What is Flux? ¶

Flux is an architecture for creating data layers in JavaScript applications. It was designed at Facebook along with the React view library. It places a focus on creating explicit and understandable update paths for your application's data, which makes tracing changes during development simpler and makes bugs easier to track down and fix.

Fluxxor is an implementation of the flux architecture pattern.

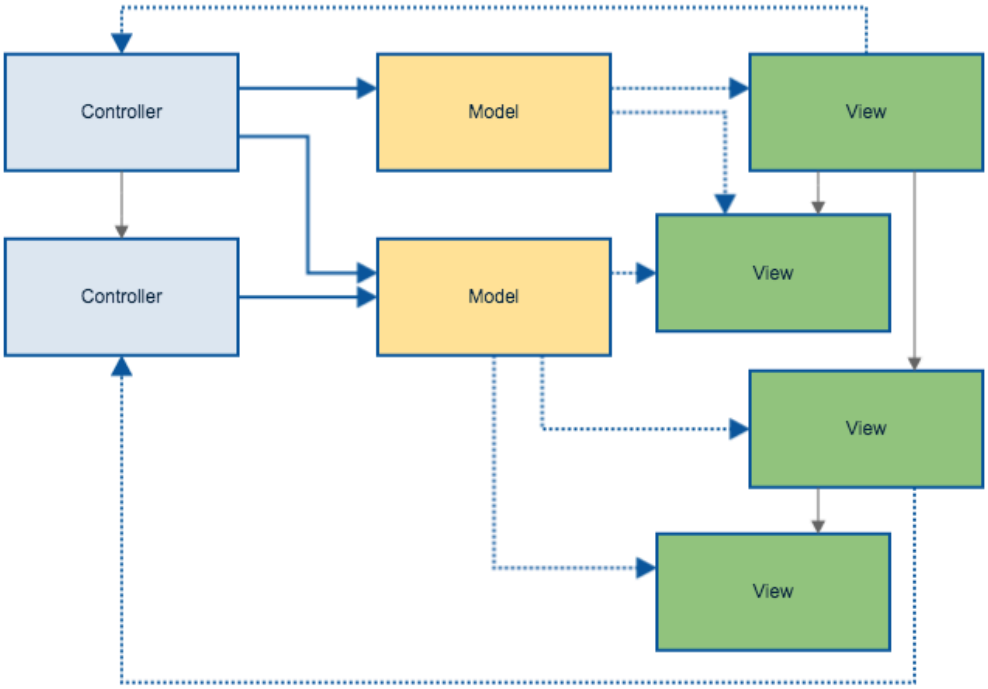
Overview

To best describe flux, we will compare it to one of the leading client-side architectures: MVC. In a client-side MVC application, a user interaction triggers code in a controller. The controller knows how to coordinate changes to one or more models by calling methods on the models. When the models change, they notify one or more views, which in turn read the new data from the models and update themselves accordingly so that the user can see that new data.



A simple MVC flow

As an MVC application grows and controllers, models, and views are added, the dependencies become more complex.



A more complex MVC flow

With the addition of just three views, one controller, and one model, the dependency graph is already harder to trace. When the user interacts with the UI, multiple branching code paths are executed, and debugging problems in application state becomes an exercise in figuring out which module (or modules) in one (or more) of these potential code paths contains a bug. In the worst cases, a user interaction will trigger updates which in turn trigger additional updates, resulting in error-prone and difficult-to-debug cascading effects along several of these, sometimes overlapping, paths.

Flux eschews this design in favor of a one-way data flow. All user interactions within a view call an *action creator*, which causes an *action* event to be emitted from a singleton *dispatcher*. The

Fork me on GitHub

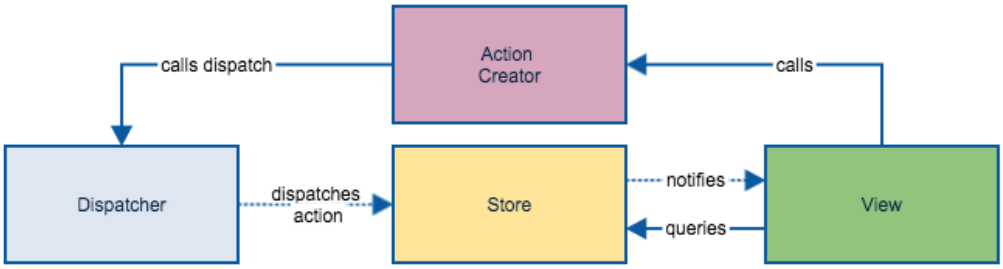
- Home
- What is Flux?
- Guides
  - Installation
  - Using with React
  - Quick-Start Guide
  - Dealing with Asynchronous Data
  - Routing
- Examples
  - Basic Todos
  - Image Carousel
  - Asynchronous Data
  - React Router
- Documentation
  - Fluxxor.Flux
  - Stores
  - Actions
  - Fluxxor.FluxMixin
  - Fluxxor.StoreWatchMixin
- FAQ
- Changelog
- View on GitHub

Support and chat on Gitter

chat on gitter

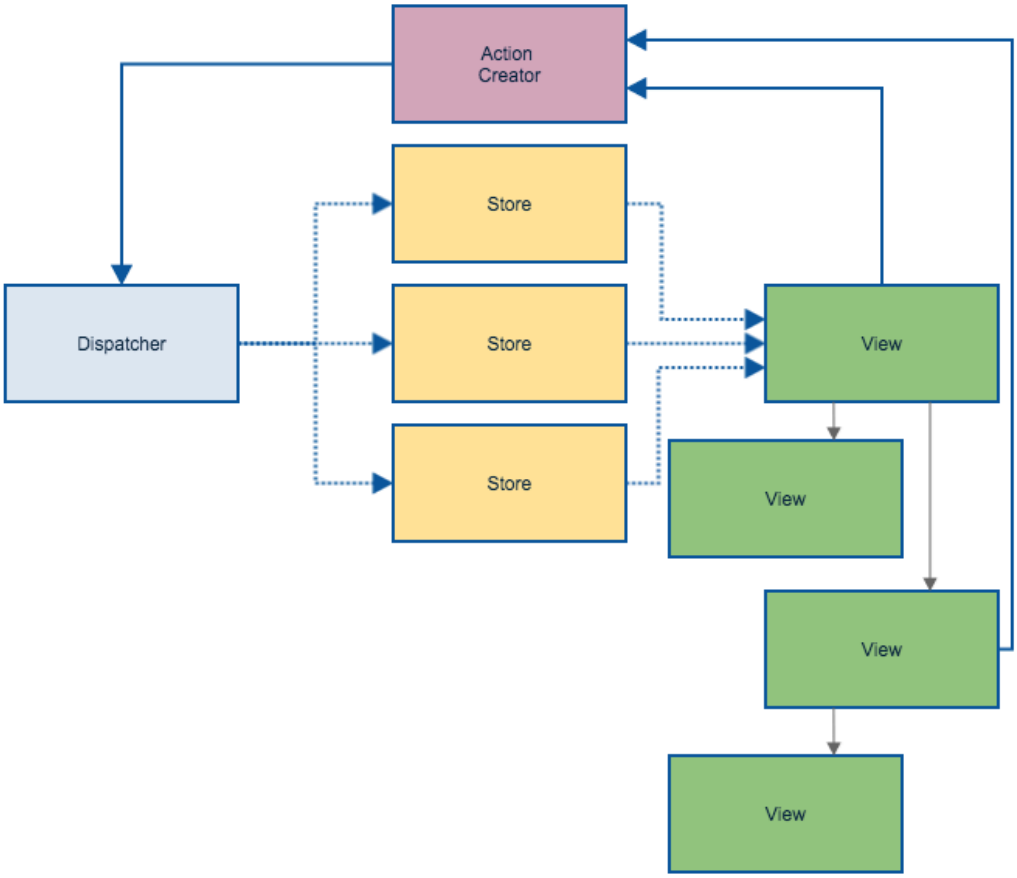
Copyright © 2014 Michelle Tilley
Licensed under the MIT license

dispatcher is a single-point-of-emission for all actions in a flux application. The action is sent from the dispatcher to *stores*, which update themselves in response to the action.



A simple flux flow

The flow doesn't change significantly for additional stores or views. The dispatcher simply sends every action to *all* the stores in the application. Note that it does not contain knowledge about how to actually update the stores—the stores themselves contain this business logic. Each store is responsible for a domain of the application, and only update themselves in response to actions.

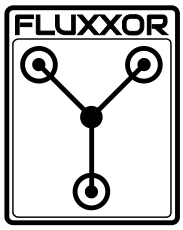


A more complex flux flow

When a store updates, it emits a change event. In many React applications, special views (known sometimes as "controller-views") are responsible for watching for this change event, reading the stores' new data, and passing that data through properties to child views. It's not uncommon in a React application for a store change event to trigger a re-render of the top-level view, effectively re-rendering the entire view hierarchy (which React handles in an efficient manner). This completely avoids complex bugs and performance problems that can arise out of trying to watch for specific property changes on models and modifying parts of views only slightly.

Key Properties

The flux architecture has a few key properties that make it unique and provide important guarantees, all of which are centered around making the flow of data explicit and easily understandable and increasing the locality of bugs (so that you don't have to hunt down many code paths to find incorrect logic). There are many flux and flux-like implementations available; these properties are, to me, most important to the flux architecture.

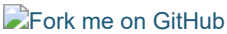


- Home
- What is Flux?
- Guides
  - Installation
  - Using with React
  - Quick-Start Guide
  - Dealing with Asynchronous Data
  - Routing
- Examples
  - Basic Todos
  - Image Carousel
  - Asynchronous Data
  - React Router
- Documentation
  - Fluxxor.Flux
  - Stores
  - Actions
  - Fluxxor.FluxMixin
  - Fluxxor.StoreWatchMixin
- FAQ
- Changelog
- View on GitHub

Support and chat on Gitter



Copyright © 2014 Michelle Tilley
Licensed under the MIT license



Enforced Synchrony

Action dispatches and their handlers inside the stores are synchronous. All asynchronous operations should trigger an action dispatch that tells the system about the result of the operation (see the async data guide for more details). While action creators can call out to asynchronous APIs, action handlers in the stores will ideally not do so. This rule makes the flow of information in the application extremely explicit; debugging errors in application state simply involves figuring out which action caused the bad state and then finding the incorrect logic that responded to that action.

Inversion of Control

Since stores update themselves internally in response to actions (rather than being updated from outside by a controller or a similar module), no other piece of the system needs to know how to modify the application's state. All the logic for updating the state is contained within the store itself. And, since stores only ever update in response to actions, and only synchronously, testing stores simply becomes a matter of putting them in an initial state, sending them an action, and testing to see if they're in the correct final state.

Semantic Actions

Since the stores need to update themselves in response to actions, the actions tend to be semantically descriptive. For example, in a flux forum application, to mark a thread as read, you might dispatch an action with a type of `MARK_THREAD_READ`. The action (and the component generating the action) doesn't know *how* to perform the update, but *describes* what it wants to happen.

Because of this property, you rarely have to change your action types, only how the stores respond to them. As long as your application has a concept of a "thread" and you have a button or other interaction that should mark a thread as read, the `MARK_THREAD_READ` action type is semantically valid.

No Cascading Actions

Flux disallows dispatching a second action as a result of dispatching an action. This helps prevent hard-to-debug cascading updates and helps you think about interactions in your application in terms of semantic actions.

Further Reading

For a more in-depth look at the Flux architecture, check out Flux Application Architecture on the official Flux site, and be sure to check out Rethinking Web App Development at Facebook from F8 on YouTube to hear Jing Chen talk more about Flux.

Ready to Try it Out?

Check out the installation and getting started guides to get up and running with Fluxxor!

See a typo? Something still not clear? Report an issue on GitHub.